

Tarea 1 — Git y GitHub: el segundo caso del detective

Ing. Fredy Ramírez Solano · ronyramsol@umes.edu.gt

Propósito: repasar por su cuenta el ciclo completo visto en clase — investigar la historia con log, blame y show; reparar en una rama; publicar con push; proponer con pull request; y deshacer con revert.

Modalidad: individual. **Tiempo estimado:** 60 a 90 minutos. **Repositorio:** su fork de tienda-la-comercial (el mismo de la clase).

ANTES DE EMPEZAR

Trabaje sobre el fork y el clon que usó en clase.

Entre al repositorio del curso en GitHub y presione **Fork**. **Importante:** desmarque la casilla "Copy the main branch only" — la necesita completa, con sus dos ramas.

1. Clone SU fork (no el repo del curso): `git clone https://github.com/SU-USUARIO/tienda-la-comercial.git`
2. Verifique su identidad: `git config --global user.name Y` `git config --global user.email` con el correo de su cuenta de GitHub.

TABLA DE COMANDOS DE REFERENCIA

Comando	Para qué sirve
<code>git clone <url></code>	Copia un repositorio remoto completo a su máquina.
<code>git status</code>	Muestra en qué rama está y en qué zona vive cada cambio.
<code>git add <archivo></code>	Prepara un cambio en el área de staging.
<code>git commit -m "mensaje"</code>	Guarda una foto permanente de lo preparado.
<code>git push</code>	Publica sus commits en su fork de GitHub.
<code>git push -u origin <rama></code>	Publica una rama nueva por primera vez.
<code>git pull</code>	Trae y fusiona los cambios del remoto.
<code>git log --oneline --graph --all</code>	Dibuja la historia completa, con todas las ramas.
<code>git log --oneline -- <archivo></code>	Historia de un solo archivo.
<code>git diff / git diff --staged</code>	Cambios sin preparar / cambios preparados.
<code>git switch <rama> / -c <rama></code>	Cambia de rama / crea una rama y se pasa a ella.
<code>git merge <rama></code>	Fusiona la rama indicada dentro de la rama actual.
<code>git merge --abort</code>	Cancela un merge en conflicto y regresa todo.
<code>git revert <hash></code>	Crea el commit inverso: deshace sin borrar historia.
<code>git blame <archivo> / -L a,b</code>	Autor, fecha y commit de cada línea (o del rango a,b).
<code>git show <hash></code>	Autor, fecha, mensaje y diff exacto de un commit.
<code>git restore <archivo></code>	Descarta cambios locales sin commit.

PARTE A — ARQUEOLOGÍA (responda con evidencia, no de memoria)

Para cada pregunta indique también QUÉ comando usó para averiguarlo.

A1. En `reportes.py` los precios aparecen con el símbolo \$ en lugar de Q. ¿Qué commit hizo ese cambio, quién es el autor y en qué fecha ocurrió?

A2. La tienda promete un descuento de mayoreo "a partir de 12 unidades", pero una venta de exactamente 12 unidades no lo recibe. Localice la línea responsable, el commit que introdujo el error, su autor, su fecha, y explique qué símbolo cambió. Puede comprobar el comportamiento con:

```
python -c "from descuentos import descuento_por_volumen; print(descuento_por_volumen(12, 10))"
0
```

A3. Copie el mensaje del commit que cambió la tasa del IVA y explique en 2 o 3 líneas por qué ese mensaje — y esa forma de trabajar — son un problema para un equipo de desarrollo.

PARTE B — REPARACIÓN CON PULL REQUEST

1. Desde `main`, cree la rama `fix/descuento-mayoreo`.
2. Corrija el error de la Parte A2 (una sola línea).
3. Verifique: el comando de comprobación debe imprimir ahora `18.0`.
4. Haga commit con un mensaje claro y publique la rama con `git push -u origin fix/descuento-mayoreo`.
5. En GitHub, abra un pull request hacia el repositorio del curso (base: repo del curso, `main`). Use esta plantilla en la descripción:

```
Qué cambié: ...
Por qué: ...
Cómo lo probé: ...
```

NOTA: su pull request va a listar también los commits que hizo en clase (nombre de tienda, lema, merge, revert). Es normal: su fork quedó adelantado respecto al repo del curso. Lo que se revisa es el commit del arreglo y la calidad de la descripción.

PARTE C — REVERT

1. En la rama `main` de su fork, revierta el commit `26a08a5` ("actualiza formato de moneda").
2. Ejecute `python reportes.py` y verifique que los montos vuelven a mostrarse en quetzales (Q).
3. Publique el cambio con `git push`.

ENTREGA

- La URL de su pull request de la Parte B.
- Las respuestas de la Parte A (A1, A2 y A3), con el comando usado en cada una. Puede escribirlas como comentario dentro de su propio pull request o en un documento breve.
- Una captura de pantalla del reporte mostrando los montos en Q (evidencia de la Parte C).

El historial de su fork cuenta la historia de su trabajo: no la borre ni la reescriba. Si algo sale mal, `git merge -abort` y `git restore` son sus salidas seguras — y siempre puede volver a leer el log.